Heart Disease Prediction using Machine Learning

This project predicts whether a patient is at risk of heart disease using medical data.

Objective: To preprocess the dataset, analyze medical features, train multiple machine learning models, compare their performance, and identify the best model.

Tools Used:

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn
- Google Colab

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("heart.csv")
df.head()
```

Show hidden output

Next steps: Generate code with df | New interactive sheet

```
df.shape
```

```
(1025, 14)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1025 non-null   int64
 1   sex       1025 non-null   int64
 2   cp        1025 non-null   int64
 3   trestbps  1025 non-null   int64
 4   chol      1025 non-null   int64
 5   fbs       1025 non-null   int64
 6   restecg   1025 non-null   int64
 7   thalach   1025 non-null   int64
 8   exang     1025 non-null   int64
 9   oldpeak   1025 non-null   float64
 10  slope     1025 non-null   int64
 11  ca        1025 non-null   int64
 12  thal      1025 non-null   int64
 13  target    1025 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

```
df.isnull().sum()
```

	0
age	0
sex	0
cp	0
trestbps	0
chol	0
fbs	0
restecg	0
thalach	0
exang	0
oldpeak	0
slope	0
ca	0
thal	0
target	0

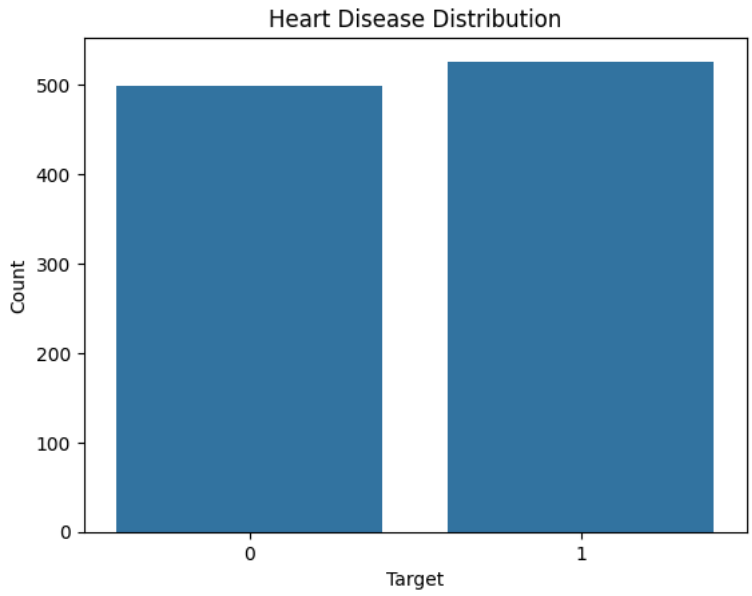
dtype: int64

```
df["target"].value_counts()
```

target	count
1	526
0	499

dtype: int64

```
sns.countplot(x="target", data=df)
plt.title("Heart Disease Distribution")
plt.xlabel("Target")
plt.ylabel("Count")
plt.show()
```



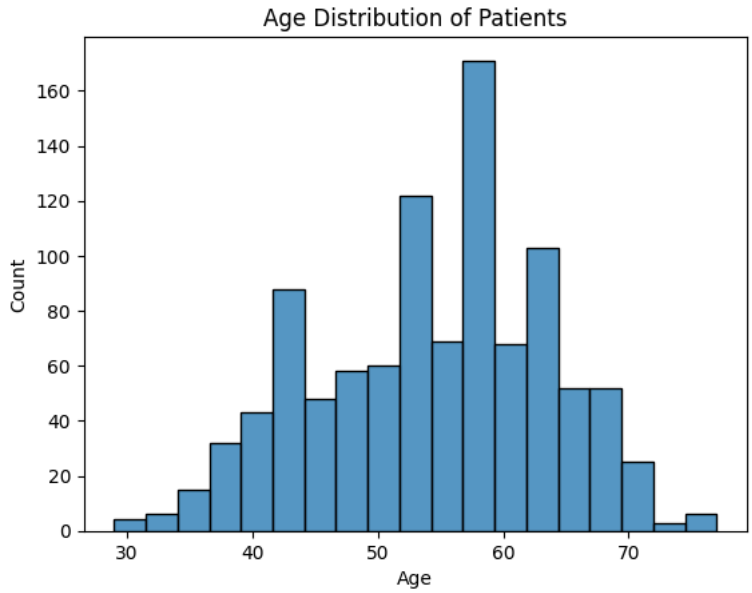
Start coding or generate with AI.

Insight 1

The dataset contains patients with and without heart disease.

The target column distribution shows that the dataset is reasonably balanced, which is good for training machine learning models without major bias.

```
sns.histplot(df["age"])
plt.title("Age Distribution of Patients")
plt.xlabel("Age")
plt.ylabel("Count")
plt.show()
```



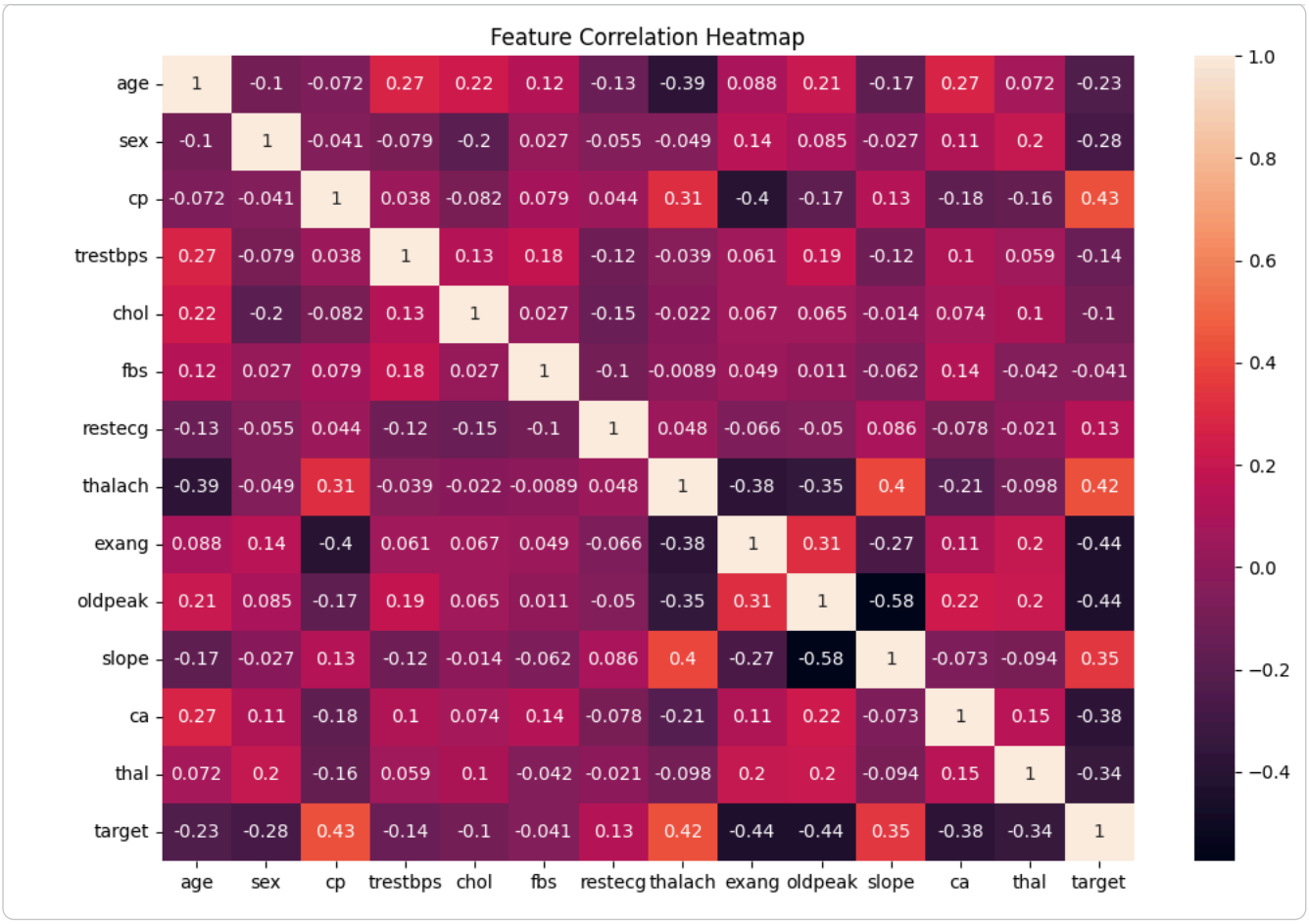

```
Start coding or generate with AI.
```

Insight 2

Most patients in the dataset belong to the middle-age group.

This suggests that heart disease risk is more common among adults, making age an important feature for machine learning prediction.

```
plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), annot=True)
plt.title("Feature Correlation Heatmap")
plt.show()
```




```
Start coding or generate with AI.
```

Insight 3

Some medical features show stronger correlation with heart disease compared to others.

This indicates that certain health factors play a bigger role in predicting heart disease and should be considered important during model training.

```
X = df.drop("target", axis=1)
y = df["target"]
```

```
print(X.head())
print(y.head())
```

```
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   52    1   0       125   212    0        1      168      0      1.0      2
1   53    1   0       140   203    1        0      155      1      3.1      0
2   70    1   0       145   174    0        1      125      1      2.6      0
3   61    1   0       148   203    0        1      161      0      0.0      2
4   62    0   0       138   294    1        1      106      0      1.9      1

   ca  thal
0   2     3
1   0     3
2   0     3
3   1     3
4   3     2
0    0
1    0
2    0
3    0
4    0
Name: target, dtype: int64
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
print(X_train.shape)
```

```
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(820, 13)
(205, 13)
(820,)
(205,)
```

```
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression()

lr.fit(X_train, y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

▾ LogisticRegression

```
LogisticRegression()
```

```
y_pred_lr = lr.predict(X_test)

print(y_pred_lr)
```

```
[1 1 0 1 0 1 0 0 1 0 1 0 1 1 0 1 0 1 1 0 1 0 1 0 1 1 1 1 0 1 1 1 1 1 1 1 1
 0 1 1 0 0 1 1 0 0 0 1 1 0 1 0 1 0 1 1 0 0 1 1 1 0 0 0 0 0 1 1 0 1 1 0 0 1
 1 1 0 1 1 1 0 0 0 0 1 0 1 0 0 1 0 0 1 1 1 1 1 1 0 0 0 0 0 1 1 0 1 0 1 0 1 1
 1 1 0 1 1 1 1 1 1 0 0 1 0 0 0 0 1 1 1 1 1 1 0 1 0 0 1 0 1 1 1 1 1 1 0 1 1 1 1
 1 0 1 0 1 1 0 0 1 1 0 0 1 1 0 0 0 0 0 0 0 1 0 1 1 0 1 1 1 0 1 1 1 0 1 1 1 1
 1 1 1 1 1 1 1 1 1 0 0 1 0 1 1 1 1 1 1 0 0]
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

accuracy_lr = accuracy_score(y_test, y_pred_lr)
precision_lr = precision_score(y_test, y_pred_lr)
recall_lr = recall_score(y_test, y_pred_lr)
f1_lr = f1_score(y_test, y_pred_lr)

print("Logistic Regression Accuracy:", accuracy_lr)
print("Logistic Regression Precision:", precision_lr)
print("Logistic Regression Recall:", recall_lr)
print("Logistic Regression F1 Score:", f1_lr)
```

```
Logistic Regression Accuracy: 0.7804878048780488
Logistic Regression Precision: 0.7377049180327869
Logistic Regression Recall: 0.8737864077669902
Logistic Regression F1 Score: 0.8
```

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(random_state=42)

rf.fit(X_train, y_train)
```

▾ RandomForestClassifier

```
RandomForestClassifier(random_state=42)
```

```
y_pred_rf = rf.predict(X_test)

print(y_pred_rf)
```

```
[1 1 0 1 0 1 0 0 1 0 1 0 1 1 0 0 0 1 1 0 0 0 0 0 0 1 1 1 0 0 0 1 0 1 1 1 0
 1 1 1 0 0 1 0 0 0 0 0 0 1 1 0 0 0 1 1 0 0 0 1 1 1 0 1 0 0 1 0 0 1 0 0 0 1
 1 1 0 0 0 1 0 0 0 0 1 0 1 0 0 0 0 0 0 1 1 1 1 0 0 0 0 1 0 0 1 0 1 0 1 0 1 0
 1 1 0 1 1 0 1 1 0 1 1 0 0 1 0 1 0 0 1 1 0 1 1 0 1 0 1 1 1 0 1 1 1 1 1 1 1 1
 0 0 0 0 1 1 0 0 0 1 0 0 1 1 0 0 1 1 0 0 1 1 0 1 1 0 1 1 1 1 0 0 1 1 0 1 0 1
 1 1 0 1 1 1 1 0 0 0 0 1 0 0 1 1 1 1 1 1 0 0]
```

```
accuracy_rf = accuracy_score(y_test, y_pred_rf)
precision_rf = precision_score(y_test, y_pred_rf)
recall_rf = recall_score(y_test, y_pred_rf)
```

```
f1_rf = f1_score(y_test, y_pred_rf)

print("Random Forest Accuracy:", accuracy_rf)
print("Random Forest Precision:", precision_rf)
print("Random Forest Recall:", recall_rf)
print("Random Forest F1 Score:", f1_rf)
```

```
Random Forest Accuracy: 0.9853658536585366
Random Forest Precision: 1.0
Random Forest Recall: 0.970873786407767
Random Forest F1 Score: 0.9852216748768473
```

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier()

knn.fit(X_train, y_train)
```

```
▾ KNeighborsClassifier
KNeighborsClassifier()
```

```
y_pred_knn = knn.predict(X_test)

print(y_pred_knn)
```

```
[1 1 0 1 0 1 0 0 1 0 1 1 1 1 0 1 0 1 1 1 0 0 1 0 0 1 1 1 1 1 0 0 0 1 1 1 0
 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 0 1 0 0 0 1 1 1 1 1 1 0 0 1 0 0 1 0 1 1 1
 1 1 0 0 0 1 1 0 0 0 1 0 1 0 1 1 0 1 1 1 1 1 0 0 0 0 1 0 0 0 0 0 0 1 0 0 1
 0 1 0 0 1 1 0 1 0 0 1 0 0 0 0 1 0 0 1 0 0 1 1 0 1 0 1 1 1 1 1 1 0 0 0 1 1
 1 0 0 0 1 1 0 0 1 1 0 1 0 0 0 0 1 0 0 0 0 1 1 1 1 0 0 1 1 0 0 1 1 0 1 1 1
 1 1 1 0 1 1 0 1 1 0 0 0 0 1 1 1 1 1 1 0 0]
```

```
accuracy_knn = accuracy_score(y_test, y_pred_knn)
precision_knn = precision_score(y_test, y_pred_knn)
recall_knn = recall_score(y_test, y_pred_knn)
f1_knn = f1_score(y_test, y_pred_knn)

print("KNN Accuracy:", accuracy_knn)
print("KNN Precision:", precision_knn)
print("KNN Recall:", recall_knn)
print("KNN F1 Score:", f1_knn)
```

```
KNN Accuracy: 0.7317073170731707
KNN Precision: 0.7307692307692307
KNN Recall: 0.7378640776699029
KNN F1 Score: 0.7342995169082126
```

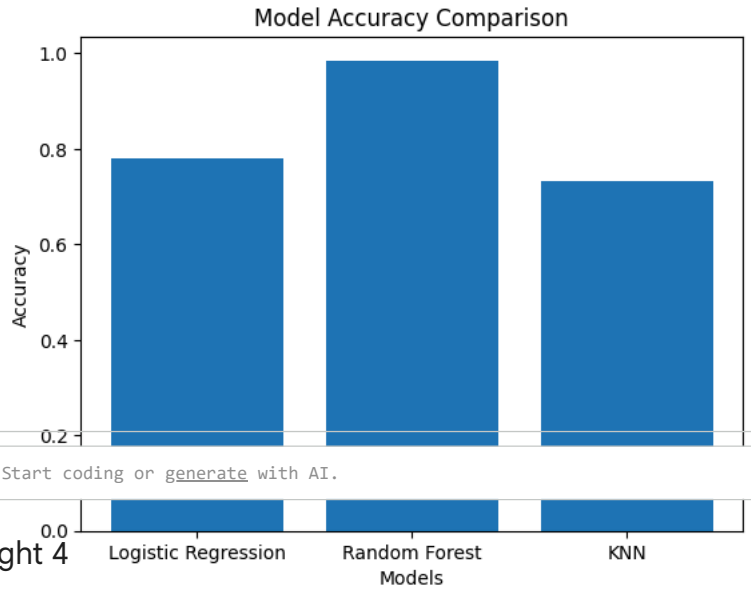
```
comparison = pd.DataFrame({
    "Model": ["Logistic Regression", "Random Forest", "KNN"],
    "Accuracy": [accuracy_lr, accuracy_rf, accuracy_knn],
    "Precision": [precision_lr, precision_rf, precision_knn],
    "Recall": [recall_lr, recall_rf, recall_knn],
    "F1 Score": [f1_lr, f1_rf, f1_knn]
})

comparison
```

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.780488	0.737705	0.873786	0.800000
1	Random Forest	0.985366	1.000000	0.970874	0.985222
2	KNN	0.731707	0.730769	0.737864	0.734300

Next steps: Generate code with comparison | New interactive sheet

```
plt.bar(comparison["Model"], comparison["Accuracy"])
plt.title("Model Accuracy Comparison")
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.show()
```




```
Start coding or generate with AI.
```

Insight 4

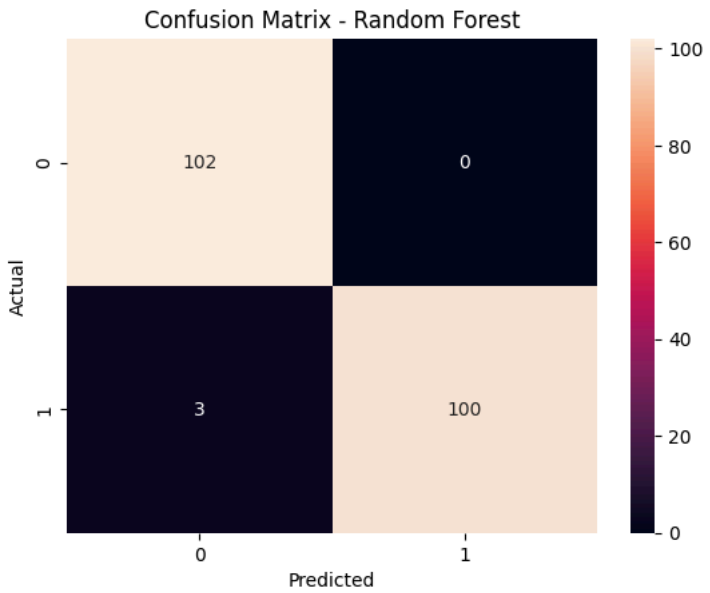
Among all three machine learning models, one model achieved better performance compared to others.

This shows that different algorithms behave differently on the same dataset, so model comparison is important before selecting the final model.

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred_rf)

sns.heatmap(cm, annot=True, fmt="d")
plt.title("Confusion Matrix - Random Forest")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```




```
Start coding or generate with AI.
```

Insight 5

The best performing model showed strong prediction performance and was able to classify patients effectively.

The confusion matrix confirmed that the model made more correct predictions than incorrect predictions, making it suitable for heart disease prediction.